

Developer Notes: Courses & Assignments Module

This document details the underlying technical architecture, relational model mechanics, and quality assurance processes for the Course and Assignment features. It serves as an architectural handoff for future developers maintaining this isolated module.

1. Architectural & Domain Logic Overview

The courses and assignments system functions as a decoupled module within the application, managing user relationships, provisioning coursework, and orchestrating proof object lifecycles.

1.1 Role Enforcements

The backend relies on user properties to isolate capabilities cleanly across API routing:

- `request.user.is_instructor`: Grants permission to instantiate courses, generate token entry arrays, alter deadlines, and read the class-wide evaluation matrix.
- `request.user.is_superuser`: Inherits all instructor permissions globally across all courses, overriding explicit ownership checks.

[!NOTE] The superuser accounts do not have a presence in the current state of proofbuddy outside of the django admin page. The permissions in these APIs are for future use and may need to be tightened or loosened as development progresses.

- Student Profiles** (`is_instructor=False, is_superuser=False`): Restricts access strictly to viewing active enrolled courses, accepting/declining invitations, and creating and working on proofs for assignments.

1.2 Isolated Ownership Checks

To prevent multi-tenant cross-contamination, endpoints that mutate data (such as `/assignments/detail/<id>`) check ownership explicitly:

```
if not request.user.is_superuser and assignment.course.instructor != request.user:
    return Response({"message": "You are not authorized..."},
                    status=status.HTTP_403_FORBIDDEN)
```

2. Database Schema & Table Layouts

2.1. Course Table

Tracks independent class meta-parameters, security validation hashes, and basic schedules.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
<code>id</code>	<code>BigAutoField</code> (Auto)	<code>False</code> / <code>False</code>	Primary Key	Auto-incremented system record index.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
name	CharField(255)	False / False		Plaintext course identifier header.
description	TextField	False / True	default=""	Optional contextual overview details.
is_active	BooleanField	False / False	default=False	Guardrail boolean dictating student visibility.
instructor	ForeignKey	True / False	Restricted to only User accounts where is_instructor is True.	Links to the instructor account that has permission to perform actions on the course.
join_code_hash	CharField(128)	True / True		SHA-256 encoded hash string matching the plain code.
join_code_expires_at	DateTimeField	True / True		Explicit timestamp expiration marker (7-day standard limit).
term	CharField(20)	False / True	The term must be a Season (Spring, Summer, Fall, Winter) followed by a four digit year (ex: Summer 2026).	Format enforcement forcing [Season] [YYYY] layout structures.
created_by	ForeignKey	False / False	Restricted to only User accounts where is_instructor is True.	Tracks row instantiation creator account.
created_at	DateTimeField	False / False	auto_now_add=True	System record creation timestamp.
updated_at	DateTimeField	False / False	auto_now=True	Last modification timestamp.

- **Implicit Join Table (Course_students):** Generated by the students ManyToManyField.

Field Name	Data Type	Description
id	BigAutoField (Auto)	Auto-incremented system record index.
course_id	Foreign Key	Id of the course containing the student.

Field Name	Data Type	Description
<code>account_id</code>	Foreign Key	Id of the student User account attached to the course.

Join Token Lifecycle

Join codes balance immediate convenience with database security through automated hashing:

- **Generation:** Utilizes `secrets.choice` to pick 8 secure characters from uppercase alphanumeric characters.
- **Storage:** The plaintext code is never stored in the database. It is stored as a `SHA-256` hex within `course.join_code_hash`.
- **Validation:** When a student enters a code via `/join_course`, the input string is encoded, run through `hashlib.sha256().hexdigest()`, and matched against active courses with unexpired join codes.

2.2. Assignment Table

The primary details about assignments created for courses. This does not store information on the proofs for each assignment.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
<code>id</code>	<code>BigAutoField (Auto)</code>	<code>False / False</code>	Primary Key	Unique coursework primary identifier.
<code>title</code>	<code>CharField(255)</code>	<code>False / False</code>		Descriptive assignment title.
<code>description</code>	<code>TextField</code>	<code>False / False</code>		Rich-text instructions or prompt criteria details (currently unused).
<code>due_date</code>	<code>DateTimeField</code>	<code>False / False</code>		Explicit calendar boundary threshold for late flags.
<code>course</code>	<code>ForeignKey</code>	<code>True / False</code>		Relational parent link directly to a <code>Course</code> record.
<code>created_by</code>	<code>ForeignKey</code>	<code>False / False</code>	Restricted to only User accounts where <code>is_instructor</code> is <code>True</code> .	Tracks row instantiation creator account.
<code>created_at</code>	<code>DateTimeField</code>	<code>False / False</code>	<code>auto_now_add=True</code>	Assignment creation timestamp.
<code>updated_at</code>	<code>DateTimeField</code>	<code>False / False</code>	<code>auto_now=True</code>	Last modification timestamp.

2.3. AssignmentProof Table

Orchestrates generic relations linking proofs of any type to assignments.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
id	BigAutoField (Auto)	False / False	Primary Key	Table entry unique reference integer.
assignment	ForeignKey	False / False	related_name=proof_items	Parent association link to the target Assignment.
order	PositiveIntegerField	False / False	default=0	Index of the proof to be listed when placed alongside other proofs for the same assignment with fallback to id if the value is equal to another proof for the same assignment.
content_type	ForeignKey	False / False	Value can be: 18 (inductionproof) or 20 (equationalproof)	System level key defining the type of proof.
object_id	PositiveIntegerField	False / False		Id of the proof in its respective table that is linked by the content_type_id.

2.4. StudentProofMapping Table

Saves background references decoupling sandboxed student workspace modifications from master versions.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
id	BigAutoField (Auto)	False / False	Primary Key	Unique mapping structural reference row index.
assignment	ForeignKey	False / False	on_delete=models.CASCADE	Maps proof to an Assignment.
student	ForeignKey	False / False	Restricted to only User accounts where is_instructor is True.	References user sandbox account workspace owner.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
template_proof_id	PositiveIntegerField	False / False		Backlink index tracker pointing to master template asset the student's proof was cloned from.
content_type	ForeignKey	False / False	Value can be: 18 (inductionproof) or 20 (equationalproof)	System level key defining the type of proof.
object_id	PositiveIntegerField	False / False		Unique pointer integer linking the student's proof on the table mapped by the content_type.
completed_at	DateTimeField	True / True		Stores the time that the first completed check passes.
• Computed Read Properties (Non-Database Fields): ◦ @property: is_late: Evaluates boolean state checking if completed_at exceeds parent assignment.due_date. ◦ @property: started_at: Pulls row initialization marker created_at dynamically from referenced polymorphic student_proof instance.				

2.5. CourseInvitation Table

Intermediate transaction storage layer staging pending registration requests sent to student accounts.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
id	BigAutoField (Auto)	False / False	Primary Key	Unique validation record primary row key.
course	ForeignKey	False / False	Must be unique combination with the student FK.	Target classroom container parent entity link.
student	ForeignKey	False / False	Linked User must have is_instructor value of False	Target User account identifier.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
status	CharField(10)	False / False	default='pending' Choices: pending, rejected	State-tracking status value flag.
sent_at	DateTimeField	False / False	auto_now_add=True	Record generation moment tracking timestamp.
updated_at	DateTimeField	False / False	auto_now=True	Status modification monitoring timestamp.

- **Meta Table Properties:**

- Validation check prevents two rows from existing with the same combination of course and student foreign keys.

2.6. AssignmentShareRequest Table

Tracks peer-to-peer instructional assignment sharing workflows. It provides a transactional mapping layer for cross-classroom coursework propagation.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
id	BigAutoField (Auto)	False / False	Primary Key	Unique validation record primary row key.
sender	ForeignKey	False / False	Restricted to User accounts where is_instructor=True.	Relational track link to the sending instructor account.
source_course	ForeignKey	False / False	related_name=sent_shares_history	The originating container environment hosting the material asset.
target_course	ForeignKey	False / False	related_name=incoming_assignment_shares	The destination container ecosystem designated to receive the asset.

Field Name	Data Type	Null / Blank	Constraints / Options	Description
staged_assignment	OneToOneField	False / False	Unique relative pointer constraint.	Holds isolated staging reference copy block before action resolution.
status	CharField(10)	False / False	default='pending' Choices: pending, accepted, rejected	State-tracking transaction parameter flag.
sent_at	DateTimeField	False / False	auto_now_add=True	Automation timestamp recorded when the model instantiates.
updated_at	DateTimeField	False / False	auto_now=True	Tracks last dynamic modification timestamp.

- **Meta Table Properties:**
 - Validation checks guarantee structural integrity by ensuring a unique combination of `source_course`, `target_course`, and `staged_assignment` fields across table rows.

Sharing Workspace Mechanics (Deep Cloning Logic)

To enforce strict tenant-isolation and protect student workspace contexts, accepting an assignment share request kicks off a nested background deep-clone process managed inside an atomic database transaction:

- **Asset Separation:** The system looks up all `AssignmentProof` metadata bindings linked to the incoming `staged_assignment`.
- **Polymorphic Deep-Cloning:** For each linked proof entry, the engine determines its table context (`equationalproof` or `inductionproof`), replicates the master proof record, copies its entire related arrays of `proof_lines` and `proof_comments`, wipes the primary keys (`pk/id = None`), and overrides ownership tracking by assigning the record to the receiving instructor. This allows the instructor to modify the base proof(s) if required and then replacing the ones one the assignment, or to simply replace a proof that was accidentally removed from a shared assignment.
- **Administrative Handoff:** Once all child records are duplicated safely in memory, the main assignment shell's metadata properties update, explicitly changing `created_by` to the recipient user. This gives the accepting instructor a totally decoupled sandbox environment, completely separated from the original author's library.

3. Quality Assurance & Verification Loop

Because the team operates without an automated CI/CD pipeline or tracking tickets, you must manually run local verification before pushing code modifications to the project lead.

3.1 Running API Unit Tests

Backend logic is validated through Django's test wrapper. Every critical permissions boundary and serializer constraint is verified automatically.

- **Execution Command:**

```
python manage.py test
```

- **Scope Coverage:** Validates role authorization failures (e.g., student triggering a course POST), payload validation limits, edge-case date expirations for code strings, and multi-record matching resolution paths.

3.2 Manual Frontend UI Test Matrix

Any visual adjustments or interface modifications within the React views must be confirmed using manual verification sweeps. Maintain a clean test db with at least one **Instructor Account** and one **Student Account**, and execute this verification matrix sequence:

Step	User Account	Dashboard Section	Action Item	Expected UI Outcome
1	Instructor	Course Page	Create Course, Toggle Active, Edit Details	Form persists, and properties save correctly.
2	Instructor	Course Settings	Generate Join Code	Display area shows alphanumeric token and correct date string.
3	Student	Course Portal	Join via Plain Token	Course appears in user list (must be an Active course).
4	Instructor	Assignment Form	Pick Proofs, Set Expiration, Set Assignment Name	Proof table appends options; assignment created cleanly.
5	Student	Assignment Deck	Click "Start Assignment"	New proof is created and the user is automatically redirected to the proof.
6	Student	Sandbox Canvas	Manipulate Lines, Complete Proof	Workspace updates smoothly and flags status as completed.
7	Instructor	Matrix Dashboard	Click Eye Icon on Assignment	Matrix grid cells color-code dynamically based on student states.

3.3 Defect Management

If any anomalies, unintended database state duplications, or user interface exceptions occur during verification sweeps, document them instantly inside the team's dedicated **#bug-reports** channel on Discord with:

1. Expected vs. actual behaviors.

2. The endpoint payload data or console stack trace.
3. Precise instructions to reproduce the anomaly locally.
4. If possible, include screenshots for front-end bugs.